Autoencoders

A one-hour lecture — theory and applications

Contents

1 Motivation: the manifold hypothesis

Real-world data sits in ambient spaces of enormous dimension but does not fill them. A 256×256 RGB image lives in $\mathbb{R}^{196,608}$, yet draw uniformly from that space and you see white noise — never a face, never a cat, never anything coherent. The set of natural images is a vanishingly small, highly structured subset.

Manifold hypothesis. Natural data concentrates near a low-dimensional manifold $\mathcal{M} \subset \mathbb{R}^D$ with intrinsic dimension $d \ll D$.

INTUITION *Generate uniform noise on your screen for a second — you will never accidentally sample a face. The set of natural images is so concentrated in pixel space that its volume relative to $\mathbb{R}^{196,608}$ is essentially zero. We are not learning a function on a high-d space; we are learning a function whose support is a thin curved sliver.*

If the hypothesis is true, then much of $\mathbb{R}^D$ is wasted. A useful representation should *discover* $\mathcal{M}$ — a map $f : \mathbb{R}^D \to \mathbb{R}^d$ that is faithful where data lives and indifferent elsewhere. There are three things to want from the code $\boldsymbol{z} = f(\boldsymbol{x})$:

1. **Compression.** Store or transmit $\boldsymbol{z}$ instead of $\boldsymbol{x}$.
2. **Inference.** Use $\boldsymbol{z}$ as input to downstream tasks — classification, regression, retrieval. Lower noise, fewer parameters.
3. **Generation.** If we know the distribution of $\boldsymbol{z}$, we can draw new samples and decode them back to $\boldsymbol{x}$.

The autoencoder is the simplest neural device that pursues all three: learn an encoder f and a decoder g such that $g(f(\boldsymbol{x})) \approx \boldsymbol{x}$ on data, and hope the resulting $\boldsymbol{z} = f(\boldsymbol{x})$ is useful.

2 History: four decades, three traditions

2.1 The three threads

Statistics (1901–). Karl Pearson introduced principal component analysis in 1901 as a way to find "lines of closest fit" to data. Hotelling generalized in 1933. This was the first formal answer to "how do I find a low-dimensional summary of high-dimensional data?"

Neuroscience (1980s–). Hopfield (1982) on associative memory, then Rumelhart and McClelland's PDP volumes, framed the question differently: a network should *auto-associate* — output a stored pattern when shown a degraded version. The backprop autoencoder of Rumelhart, Hinton, and Williams (1986) is the direct neural realization.

Information theory (1948–). Shannon's rate–distortion theory asks: given a budget of R bits per source symbol, what is the smallest distortion achievable? The autoencoder bottleneck is a concrete (if heuristic) implementation of this trade-off.

2.2 Key milestones

- **1936.** Eckart & Young prove that the truncated SVD is the best low-rank approximation in Frobenius norm. The cornerstone result everything linear rests on.
- **1988.** Bourlard & Kamp prove that a linear autoencoder with squared loss recovers the PCA subspace. The first formal bridge between neural networks and classical statistics.
- **1989.** Baldi & Hornik strengthen this: every local minimum of the linear autoencoder is global.
- **1996.** Olshausen & Field — sparse coding of natural images yields V1-like receptive fields. Representation learning starts being taken seriously.
- **2006.** Hinton & Salakhutdinov stack restricted Boltzmann machines to pretrain deep autoencoders. *Science* paper that helped restart the deep-learning era.
- **2008–11.** Vincent et al. on denoising autoencoders. Rifai et al. on contractive autoencoders. The regularized-AE programme.
- **2013–14.** Kingma & Welling, and independently Rezende et al., introduce the variational autoencoder. Variational inference meets neural nets via the reparameterization trick.
- **2017.** Van den Oord et al. introduce VQ-VAE: discrete latent codes.
- **2022.** Latent diffusion (Rombach et al.) and masked autoencoders (He et al.) put AEs at the centre of modern generative and self-supervised models.

3 Linear theory: PCA, Eckart–Young, Bourlard–Kamp

3.1 The linear autoencoder

$$\begin{aligned} \boldsymbol{z} &= \boldsymbol{W}_1 \boldsymbol{x}, \qquad \boldsymbol{W}_1 \in \mathbb{R}^{d \times D} \\ \hat{\boldsymbol{x}} &= \boldsymbol{W}_2 \boldsymbol{z}, \qquad \boldsymbol{W}_2 \in \mathbb{R}^{D \times d} \\ \mathcal{L} &= \mathbb{E}\, \|\boldsymbol{x} - \boldsymbol{W}_2 \boldsymbol{W}_1 \boldsymbol{x}\|^2 \end{aligned}$$

This is the most stripped-down autoencoder imaginable: linear encoder, linear decoder, squared loss. Centre the data so $\mathbb{E}[\boldsymbol{x}] = 0$. The question is: what does the optimum look like?

3.2 The Eckart–Young theorem

Theorem 3.1 (Eckart–Young 1936; Mirsky 1960)**.** *Let $\boldsymbol{X} \in \mathbb{R}^{n \times D}$ have singular value decomposition $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$ with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. For any rank-$d$ matrix $\tilde{\boldsymbol{X}}$,*

$$\|\boldsymbol{X} - \tilde{\boldsymbol{X}}\|_{\mathrm{F}}^2 \;\geq\; \sum_{i=d+1}^{r} \sigma_i^2,$$

with equality iff $\tilde{\boldsymbol{X}} = \boldsymbol{U}_d \boldsymbol{\Sigma}_d \boldsymbol{V}_d^\top$, the truncated SVD. The same bound holds in spectral norm (Mirsky).

Proof sketch. The squared Frobenius norm $\|\boldsymbol{X} - \tilde{\boldsymbol{X}}\|_{\mathrm{F}}^2 = \operatorname{tr}\big((\boldsymbol{X} - \tilde{\boldsymbol{X}})^\top (\boldsymbol{X} - \tilde{\boldsymbol{X}})\big)$ is unitarily invariant. So pre- and post-multiplying by orthogonal matrices does not change the value:

$$\|\boldsymbol{X} - \tilde{\boldsymbol{X}}\|_{\mathrm{F}} = \|\boldsymbol{U}^\top (\boldsymbol{X} - \tilde{\boldsymbol{X}}) \boldsymbol{V}\|_{\mathrm{F}} = \|\boldsymbol{\Sigma} - \boldsymbol{U}^\top \tilde{\boldsymbol{X}} \boldsymbol{V}\|_{\mathrm{F}}.$$

Write $\boldsymbol{B} = \boldsymbol{U}^\top \tilde{\boldsymbol{X}} \boldsymbol{V}$; this is also rank d. The problem becomes: among rank-d matrices $\boldsymbol{B}$, minimize $\|\boldsymbol{\Sigma} - \boldsymbol{B}\|_{\mathrm{F}}^2 = \sum_{i,j} (\boldsymbol{\Sigma}_{ij} - B_{ij})^2$. Since $\boldsymbol{\Sigma}$ is diagonal, the off-diagonal contribution to the loss is $\sum_{i \neq j} B_{ij}^2 \geq 0$, minimized by $B_{ij} = 0$ off-diagonal. The on-diagonal contribution is $\sum_i (\sigma_i - B_{ii})^2$. Subject to $\boldsymbol{B}$ having rank d, at most d of the B_{ii} can be non-zero, so we should set $B_{ii} = \sigma_i$ for the d largest singular values and zero elsewhere. The residual is $\sum_{i>d} \sigma_i^2$, achieved by $\tilde{\boldsymbol{X}} = \boldsymbol{U}_d \boldsymbol{\Sigma}_d \boldsymbol{V}_d^\top$. □

> **INTUITION** *In the SVD basis the problem is trivial: you have a diagonal matrix and need to zero out all but d entries to make it rank d. The optimal choice is to keep the biggest ones. The whole proof is just* the right change of basis exposes the answer. *This is a recurring theme in machine learning — find the basis in which the problem is diagonal.*

3.3 Corollary: PCA solves the linear autoencoder

Theorem 3.2 (Bourlard–Kamp 1988; Baldi–Hornik 1989)**.** *The optimum of*

$$\min_{\boldsymbol{W}_1, \boldsymbol{W}_2} \ \mathbb{E}\,\|\boldsymbol{x} - \boldsymbol{W}_2\boldsymbol{W}_1\boldsymbol{x}\|^2$$

is achieved when $\boldsymbol{W}_2\boldsymbol{W}_1$ projects orthogonally onto the span of the top d eigenvectors of $\boldsymbol{\Sigma}_{xx} = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$. Every local minimum is global.

3.4 Identifiability — a permanent caveat

The objective is invariant under $\boldsymbol{W}_1 \to \boldsymbol{A}^{-1}\boldsymbol{W}_1$, $\boldsymbol{W}_2 \to \boldsymbol{W}_2\boldsymbol{A}$ for any invertible $\boldsymbol{A} \in \mathbb{R}^{d\times d}$. The *span* of the columns of $\boldsymbol{W}_2$ is identified — it is the principal subspace — but individual latent units are not the principal components.

> **GOTCHA** *People look at TensorBoard plots showing "feature 7 encodes age" and think the model has discovered ageing. Sometimes it has; usually it has discovered a random linear combination that happens to correlate with age in the test batch. Retrain with a different seed and a different combination will correlate. The model never agreed to give you axis-aligned semantic factors.*

Remark 3.3. The same non-identifiability afflicts nonlinear autoencoders, and worse: any diffeomorphism of the latent space leaves the data manifold intact. This is the formal obstruction in Locatello et al. (2019) and the reason all disentanglement methods require some inductive bias or supervision.

> **EXAMPLE** **When does PCA fail? The Swiss roll**
>
> The Swiss roll is the canonical example where linear methods break. Sample $n = 2000$ points on the 2D surface
>
> $$\boldsymbol{x}(t, s) = \big(t\cos t,\ s,\ t\sin t\big) + 0.1\,\boldsymbol{\varepsilon}, \qquad t \in [3\pi/2,\ 9\pi/2],\ s \in [0, 21].$$
>
> Intrinsic dimension: $d = 2$. Ambient dimension: $D = 3$.
>
> **What PCA gives you.** The top-2 principal components capture roughly $\sigma_1^2 + \sigma_2^2 \approx 88\%$ of the variance — the plane through the roll. But this plane *flattens* the spiral: points from the inner coil collide with points from the outer coil. Pairwise distances are catastrophically wrong.
>
> **What a nonlinear AE gives you.** An MLP encoder/decoder with $D = 3 \to 16 \to 2 \to 16 \to 3$, trained for ∼5000 steps with Adam at 10^{-3}, learns to "unroll" the spiral. Reconstruction MSE drops from ~ 7 (PCA) to < 0.05 (AE) on this dataset.
>
> ```
> class AE(nn.Module):
>     def __init__(self, D=3, d=2, h=16):
>         super().__init__()
>         self.enc = nn.Sequential(nn.Linear(D,h), nn.ReLU(), nn.Linear(h,d))
>         self.dec = nn.Sequential(nn.Linear(d,h), nn.ReLU(), nn.Linear(h,D))
>     def forward(self, x):
>         return self.dec(self.enc(x))
>
> loss = ((model(x) - x)**2).mean()   # Gaussian likelihood => MSE
> ```
>
> **Reality check.** Real images are far harder than the Swiss roll: the manifold is high-dimensional, non-uniform, and full of holes. But the failure mode is the same — PCA on 32×32 natural images keeps about 95% of variance in the top 50 components, yet the reconstructions look like smeared low-frequency blobs. A small convolutional AE with the same latent budget reconstructs crisply.

WHY THIS WORKS **Linear theory in one paragraph.**

A linear autoencoder with squared loss has *no useful local minima*: every local minimum is global, and the global minimum recovers the principal-subspace projection. This is surprising because the loss is non-convex in $(\boldsymbol{W}_1, \boldsymbol{W}_2)$ jointly — yet the geometry of the problem rules out non-global traps. The price you pay is non-identifiability: the model identifies the right subspace but not a preferred basis within it. Nonlinear AEs inherit both properties — better fitting power because they can curve to track the data manifold, and worse identifiability because diffeomorphism is a much larger symmetry group than orthogonal rotation. The whole subsequent lecture is about adding inductive biases that break this

symmetry: regularization (DAE, contractive) breaks it geometrically, prior assumptions (VAE) break it probabilistically, discreteness (VQ-VAE) breaks it combinatorially.

4 Nonlinear autoencoders: architecture and loss

4.1 Architecture

Replace $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ with neural networks. The encoder $f_\theta : \mathbb{R}^D \to \mathbb{R}^d$ and decoder $g_\phi : \mathbb{R}^d \to \mathbb{R}^D$ are parametrised by weights θ, ϕ. The reconstruction is $\hat{\boldsymbol{x}} = g_\phi(f_\theta(\boldsymbol{x}))$.

4.2 Loss = negative log-likelihood

If the decoder defines $p_\phi(\boldsymbol{x} \mid \boldsymbol{z})$, the reconstruction loss is $-\log p_\phi(\boldsymbol{x} \mid \boldsymbol{z})$. Different noise models give different losses:

Data	Noise model	Loss
continuous	Gaussian	$\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$ (MSE)
pixels in $[0,1]$	Bernoulli	$-\sum x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)$
categorical	categorical	cross-entropy
counts	Poisson	$\hat{x} - x \log \hat{x}$
heavy-tail	Laplace	$\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_1$

INTUITION *Why "averaging over modes" produces blur, made concrete: if the same input admits two equally good reconstructions — say, a vertical edge or a slightly shifted vertical edge — the MSE optimum is their pixel-wise mean, which is a wider blurry edge. The optimum of a unimodal Gaussian likelihood is the conditional mean; if the true conditional is bimodal, the mean lives in a low-density valley between the modes. This is why GANs and diffusion look sharper than vanilla VAEs at equal capacity — they do not have this implicit unimodality assumption.*

4.3 Bottleneck dimension

Undercomplete. The bottleneck forces compression. The model cannot copy $\boldsymbol{x}$ verbatim. By Theorem 3.2, the linear case fits the PCA subspace; the nonlinear case fits a curved d-dimensional manifold.

Overcomplete. The identity map $g \circ f = \mathrm{id}$ is trivially optimal, so no learning happens without regularization. We will see in Section 5 how to make overcomplete codes useful.

EXAMPLE **The simplest working autoencoder: MNIST**

The canonical pedagogical setting. Each MNIST image is 28×28, so $D = 784$. A latent of $d = 32$ is generous — enough to encode digit identity, stroke width, slant, and a few residual modes.

Architecture and dimensions. Convolutional encoder/decoder, total $\approx$ 230k parameters:

input $\boldsymbol{x}$: $(B, 1, 28, 28)$ $\to$ Conv+ReLU: $(B, 32, 14, 14)$ $\to$ Conv+ReLU: $(B, 64, 7, 7)$ $\to$ flatten + Linear: $\boldsymbol{z} \in \mathbb{R}^{32}$.
Decoder mirrors back to $(B, 1, 28, 28)$.

Training. Batch size $B = 128$, Adam at 10^{-3}, BCE loss (pixels in $[0,1]$). After 20 epochs ($\sim$3 min on a single GPU):

Quantity	Value
Train reconstruction BCE	≈ 0.085 nats/pixel
Test reconstruction BCE	≈ 0.088 nats/pixel
Mean per-image MSE	≈ 0.010
Effective rank of $J_g(\boldsymbol{z})$ (median)	≈ 11

What you read off this. The effective decoder rank of $\sim$11 — far below $d = 32$ — is the empirical intrinsic dimension. MNIST has roughly 10 modes (the digits) plus a few stylistic axes; the AE discovers this without supervision.

```
# core training step -- Bernoulli decoder, BCE loss
for x, _ in loader:                      # x in [0,1], shape (B, 1, 28, 28)
    z = encoder(x)                       # (B, 32)
    x_hat = decoder(z)                   # (B, 1, 28, 28), sigmoid output
    loss = F.binary_cross_entropy(x_hat, x, reduction='mean')
    opt.zero_grad(); loss.backward(); opt.step()
```

The point of the exercise. The MNIST AE works, but $\boldsymbol{z}$ has no usable structure — linearly interpolating between the encodings of a 3 and an 8 produces a mush of overlaid strokes, not a smooth morph. That is the failure mode the VAE will fix in §8.3.

5 Regularized autoencoders and the manifold view

5.1 Sparse autoencoders

Add a penalty on the activations of latent units:

$$\mathcal{L}_{\text{SAE}} = \mathbb{E}\,\|\boldsymbol{x} - g(f(\boldsymbol{x}))\|^2 + \lambda\,\Omega(\boldsymbol{z}),$$

with $\Omega(\boldsymbol{z}) = \|\boldsymbol{z}\|_1$ or a KL divergence between the average activation and a low Bernoulli prior. Encourages a few units active per input, like sparse coding.

5.2 Denoising autoencoders

Vincent et al. (2008): corrupt the input with $\tilde{\boldsymbol{x}} \sim q(\tilde{\boldsymbol{x}} \mid \boldsymbol{x})$, reconstruct the clean $\boldsymbol{x}$. The objective is

$$\mathcal{L}_{\text{DAE}} = \mathbb{E}_{\boldsymbol{x},\tilde{\boldsymbol{x}}}\,\|\boldsymbol{x} - g(f(\tilde{\boldsymbol{x}}))\|^2.$$

Common corruptions: additive Gaussian $\tilde{\boldsymbol{x}} = \boldsymbol{x} + \sigma\boldsymbol{\varepsilon}$, salt-and-pepper noise, masking (set a random subset of coordinates to zero).

5.3 Contractive autoencoders

Rifai et al. (2011): add a penalty on the encoder Jacobian,

$$\mathcal{L}_{\text{CAE}} = \mathbb{E}\,\|\boldsymbol{x} - g(f(\boldsymbol{x}))\|^2 + \lambda\|J_f(\boldsymbol{x})\|_{\text{F}}^2,$$

where $J_f(\boldsymbol{x}) = \partial f/\partial \boldsymbol{x} \in \mathbb{R}^{d\times D}$. The penalty makes the encoder locally invariant — derivatives along arbitrary directions in $\mathbb{R}^D$ are suppressed. The decoder must oppose this on the tangent directions to actually fit the data.

5.4 The manifold view

At each $\boldsymbol{x} \in \mathcal{M}$, the ambient tangent space $\mathbb{R}^D$ splits as

$$\mathbb{R}^D = T_{\boldsymbol{x}}\mathcal{M} \oplus N_{\boldsymbol{x}}\mathcal{M},$$

into directions *along* the manifold and *off* the manifold. A good autoencoder should be:

- *Sensitive* to perturbations along $T_{\boldsymbol{x}}\mathcal{M}$ — those change the data.
- *Insensitive* to perturbations along $N_{\boldsymbol{x}}\mathcal{M}$ — those are noise.

Geometric reading. The encoder Jacobian J_f has d rows. At the optimum, its row space approximates $T_{\boldsymbol{x}}\mathcal{M}$, viewed in $\mathbb{R}^D$; equivalently, the singular spectrum of J_f should show a clean gap at the intrinsic dimension d.

INTUITION *Picture a piece of paper crumpled in three dimensions. Moving* along *the paper (sliding a finger over it) changes which point on the paper you are at — that is the tangent direction. Moving* off *the paper (lifting your finger up) takes you to a point that is not on the paper at all — that is the normal direction. A good autoencoder should notice tangent moves (different inputs) and not notice normal moves (same input plus noise). Regularization is the mechanism that enforces "not noticing".*

The contractive penalty makes this explicit by directly shrinking $\|J_f\|_{\text{F}}$. The denoising procedure achieves the same effect implicitly — if a small perturbation of $\boldsymbol{x}$ must still decode to $\boldsymbol{x}$, then $g \circ f$ is locally contractive (Alain & Bengio 2014).

EXAMPLE **Denoising in practice: BSD68 / Natural images**

Setup. BSD68, a standard image-denoising benchmark: $D = 481 \times 321 \times 1$ greyscale images corrupted with Gaussian noise at fixed $\sigma \in \{15, 25, 50\}$ on a $[0, 255]$ scale. Train a CNN denoiser to map $\tilde{\boldsymbol{x}} \mapsto \boldsymbol{x}$.

Architecture (DnCNN, Zhang et al. 2017). A pure denoising autoencoder, but *residual*: predict the noise, not the image. 17 conv layers, no pooling, $\approx$ 560k parameters.

$$\tilde{\boldsymbol{x}} \xrightarrow{\text{17 conv+BN+ReLU}} \hat{\eta}, \qquad \hat{\boldsymbol{x}} = \tilde{\boldsymbol{x}} - \hat{\eta}.$$

Loss: $\mathcal{L} = \frac{1}{2}\|\hat{\eta} - (\tilde{\boldsymbol{x}} - \boldsymbol{x})\|^2$.

Numbers. On BSD68 with $\sigma = 25$:

Method	PSNR (dB)
Noisy input	20.18
BM3D (classical baseline)	28.57
DnCNN (denoising AE)	29.23

A simple DAE beats 15 years of careful signal-processing engineering by ≈ 0.7 dB. Doesn't sound like much — it's a halving of MSE in linear units.

Why residual prediction? The residual $\boldsymbol{x} - \tilde{\boldsymbol{x}} = -\sigma\boldsymbol{\varepsilon}$ is approximately white noise; the network is asked to estimate something close to mean-zero with bounded scale. Direct $\tilde{\boldsymbol{x}} \mapsto \boldsymbol{x}$ regression would have the network learn the identity map plus a small correction, which is numerically wasteful. This is exactly the parameterization $g(\tilde{\boldsymbol{x}}) = \tilde{\boldsymbol{x}} + \sigma^2 \boldsymbol{s}_\theta(\tilde{\boldsymbol{x}})$ that connects DAEs to score matching — see the next section.

```
# DAE training loop -- residual parameterization
for x in loader:                          # clean image
    sigma = 25 / 255.0
    eps = torch.randn_like(x)
    x_tilde = x + sigma * eps             # corrupt
    eta_hat = net(x_tilde)                # predict noise
    loss = 0.5 * ((eta_hat - sigma * eps)**2).mean()
    opt.zero_grad(); loss.backward(); opt.step()
```

6 Score matching and the denoising-diffusion bridge

6.1 Score matching (Hyvärinen 2005)

Definition 6.1. The *score* of a density p is $\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$.

Suppose we want to learn a model $\boldsymbol{s}_\theta(\boldsymbol{x})$ matching the true score. The natural loss is the squared error

$$J(\theta) = \tfrac{1}{2}\mathbb{E}_p \, \|\boldsymbol{s}_\theta(\boldsymbol{x}) - \nabla \log p(\boldsymbol{x})\|^2. \tag{1}$$

This appears intractable: we do not know p. But Hyvärinen's trick eliminates $\nabla \log p$ using integration by parts.

Theorem 6.2 (Hyvärinen identity)**.** *Under regularity conditions (decay of p and $\boldsymbol{s}_\theta$ at infinity),*

$$J(\theta) = \mathbb{E}_p\big[\tfrac{1}{2}\|\boldsymbol{s}_\theta(\boldsymbol{x})\|^2 + \operatorname{tr}(\nabla_{\boldsymbol{x}} \boldsymbol{s}_\theta(\boldsymbol{x}))\big] + \text{const}. \tag{2}$$

The constant does not depend on θ.

Proof sketch. Expand the square in (1):

$$J(\theta) = \tfrac{1}{2}\mathbb{E}_p\|\boldsymbol{s}_\theta\|^2 - \mathbb{E}_p[\boldsymbol{s}_\theta^\top \nabla \log p] + \text{const}.$$

For the cross term, write $\nabla \log p = (\nabla p)/p$ and integrate:

$$\mathbb{E}_p[\boldsymbol{s}_\theta^\top \nabla \log p] = \int \boldsymbol{s}_\theta(\boldsymbol{x})^\top \nabla p(\boldsymbol{x}) \, d\boldsymbol{x}.$$

Integrate by parts coordinate by coordinate (boundary terms vanish by decay):

$$= -\int p(\boldsymbol{x})\, \nabla \cdot \boldsymbol{s}_\theta(\boldsymbol{x})\, d\boldsymbol{x} = -\mathbb{E}_p[\operatorname{tr}(\nabla \boldsymbol{s}_\theta)].$$

Substituting gives (2). □

6.2 Denoising score matching (Vincent 2011)

Smooth the data density by convolving with a Gaussian kernel:

$$q_\sigma(\tilde{\boldsymbol{x}} \mid \boldsymbol{x}) = \mathcal{N}(\tilde{\boldsymbol{x}}; \boldsymbol{x}, \sigma^2 I), \qquad p_\sigma(\tilde{\boldsymbol{x}}) = \int q_\sigma(\tilde{\boldsymbol{x}} \mid \boldsymbol{x})\, p(\boldsymbol{x})\, d\boldsymbol{x}.$$

The score of the kernel itself is in closed form:

$$\nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}} \mid \boldsymbol{x}) = \frac{\boldsymbol{x} - \tilde{\boldsymbol{x}}}{\sigma^2}.$$

Theorem 6.3 (Vincent 2011)**.** *Up to an additive constant in θ,*

$$\tfrac{1}{2}\mathbb{E}_{p_\sigma} \|\boldsymbol{s}_\theta(\tilde{\boldsymbol{x}}) - \nabla \log p_\sigma(\tilde{\boldsymbol{x}})\|^2 \;=\; \tfrac{1}{2}\mathbb{E}_{p,\, q_\sigma} \left\| \boldsymbol{s}_\theta(\tilde{\boldsymbol{x}}) - \frac{\boldsymbol{x} - \tilde{\boldsymbol{x}}}{\sigma^2} \right\|^2. \tag{3}$$

Proof sketch. Expand the left-hand square. The model-norm and constant terms agree on both sides. The cross term on the left is

$$-\mathbb{E}_{p_\sigma}\left[\boldsymbol{s}_\theta(\tilde{\boldsymbol{x}})^\top \nabla \log p_\sigma(\tilde{\boldsymbol{x}})\right] = -\int \boldsymbol{s}_\theta(\tilde{\boldsymbol{x}})^\top \nabla p_\sigma(\tilde{\boldsymbol{x}})\, d\tilde{\boldsymbol{x}}.$$

Differentiate under the integral: $\nabla p_\sigma(\tilde{\boldsymbol{x}}) = \int \nabla q_\sigma(\tilde{\boldsymbol{x}} \mid \boldsymbol{x})\, p(\boldsymbol{x})\, d\boldsymbol{x}$. Therefore the cross term equals

$$-\mathbb{E}_{p,q_\sigma}\left[\boldsymbol{s}_\theta(\tilde{\boldsymbol{x}})^\top \nabla \log q_\sigma(\tilde{\boldsymbol{x}} \mid \boldsymbol{x})\right] = -\mathbb{E}_{p,q_\sigma}\left[\boldsymbol{s}_\theta(\tilde{\boldsymbol{x}})^\top \frac{\boldsymbol{x} - \tilde{\boldsymbol{x}}}{\sigma^2}\right],$$

which matches the cross term obtained by expanding the right-hand side of (3). □

6.3 Reading the equivalence

Reparametrise the denoising autoencoder as $g(\tilde{\boldsymbol{x}}) = \tilde{\boldsymbol{x}} + \sigma^2 \boldsymbol{s}_\theta(\tilde{\boldsymbol{x}})$. The standard DAE objective

$$\mathbb{E}\,\|\boldsymbol{x} - g(\tilde{\boldsymbol{x}})\|^2 = \mathbb{E}\,\|\boldsymbol{x} - \tilde{\boldsymbol{x}} - \sigma^2 \boldsymbol{s}_\theta(\tilde{\boldsymbol{x}})\|^2 = \sigma^4\, \mathbb{E} \left\| \boldsymbol{s}_\theta(\tilde{\boldsymbol{x}}) - \frac{\boldsymbol{x} - \tilde{\boldsymbol{x}}}{\sigma^2} \right\|^2$$

is exactly the right-hand side of (3) up to the factor σ^4.

Punchline. The optimal denoiser at noise level σ satisfies

$$g^*(\tilde{\boldsymbol{x}}) - \tilde{\boldsymbol{x}} = \sigma^2\, \nabla_{\tilde{\boldsymbol{x}}} \log p_\sigma(\tilde{\boldsymbol{x}}).$$

A denoising autoencoder learns the score of the noise-smoothed data distribution.

INTUITION *The score $\nabla \log p(\boldsymbol{x})$ at a point $\boldsymbol{x}$ is a vector that points toward higher data density. Think of it as the wind blowing toward the data manifold from wherever you happen to be standing. A noisy sample is upwind; following the score brings you back. The DAE residual $g(\tilde{\boldsymbol{x}}) - \tilde{\boldsymbol{x}}$ is the wind direction scaled by σ^2 — larger noise levels need larger pushes back.*

HISTORY *Vincent's denoising-score-matching result was published in 2011. It took until 2019–2020 for it to become the foundation of state-of-the-art image generation. Why the nine-year gap? The single-noise-level DAE is good at denoising; it is bad at generation because sampling from p_σ for fixed small σ requires a Langevin chain that mixes terribly between modes. Song and Ermon's insight (NCSN, 2019) was to train one network across many noise levels and anneal from large to small — the large-σ score is easy to follow (the distribution is nearly Gaussian) and provides a smooth path into the small-σ regime where modes are sharp. Ho et al.'s DDPM (2020) wrapped this in a clean*

discrete-time formulation. The math was there for nine years; the engineering insight took the rest.

EXAMPLE **DDPM: the denoising AE generalized to a continuum of noise**

The bridge in concrete terms. A DDPM (Ho, Jain, Abbeel 2020) is a denoising autoencoder trained at every noise level σ_t, $t = 1, \dots, T$. The same network $\boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t)$ predicts the noise that was added; the score follows from the reparameterization

$$\boldsymbol{s}_\theta(\boldsymbol{x}_t, t) = -\boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t)/\sigma_t.$$

Forward diffusion. Define a variance schedule $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$. For DDPM-CelebA: $T = 1000$, linear $\beta_t \in [10^{-4}, 0.02]$, giving $\bar{\alpha}_T \approx 4 \times 10^{-5}$ — $\boldsymbol{x}_T$ is essentially pure noise. Closed-form forward kernel:

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, I).$$

Training loss — one line. Sample $(\boldsymbol{x}_0, t, \boldsymbol{\varepsilon})$ uniformly and minimize

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t, \boldsymbol{x}_0, \boldsymbol{\varepsilon}} \left\| \boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t) \right\|^2.$$

This is exactly the denoising score-matching loss from Theorem 6.3, summed over noise levels. No new mathematics.

```
# DDPM training step -- exactly the DAE loss, sampled over noise levels t
for x0 in loader:
    t = torch.randint(0, T, (x0.size(0),))    # uniform noise level
    eps = torch.randn_like(x0)
    abar = alpha_bar[t].view(-1,1,1,1)
    xt = abar.sqrt() * x0 + (1 - abar).sqrt() * eps
    eps_pred = net(xt, t)                     # conditioned on level
    loss = ((eps - eps_pred)**2).mean()
    opt.zero_grad(); loss.backward(); opt.step()
```

Sampling. Reverse the chain: $\boldsymbol{x}_T \sim \mathcal{N}(0, I)$, then T steps of $\boldsymbol{x}_{t-1} = \boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t) + \sigma_t \boldsymbol{z}$, $\boldsymbol{z} \sim \mathcal{N}(0, I)$, where $\boldsymbol{\mu}_\theta$ is a closed-form linear function of $\boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t)$. *Following the learned score field from noise to data*, exactly as the punchline of §6.3 predicted.

Real-system numbers.

Model	Resolution	Params	Sampling steps
DDPM (Ho+ 2020), CIFAR-10	32×32	36M	1000
ADM (Dhariwal-Nichol 2021), ImageNet	256×256	553M	250
Stable Diffusion v1.5	512×512	860M	25–50 (in latent)

The training loss is one line of denoising AE; the difference between research code and Stable Diffusion is engineering — noise schedules, conditioning, U-Net architecture, classifier-free guidance, and latent-space compression via a VAE.

7 Information-theoretic view

7.1 Rate–distortion

Let $\boldsymbol{x}$ be a random source. A stochastic encoder $q(\boldsymbol{z} \mid \boldsymbol{x})$ maps it to a code $\boldsymbol{z}$, and a decoder reconstructs $\hat{\boldsymbol{x}}(\boldsymbol{z})$. Define

$$R = I(\boldsymbol{x}; \boldsymbol{z}) \ \text{(rate, in nats)}, \qquad D = \mathbb{E}\left[d(\boldsymbol{x}, \hat{\boldsymbol{x}}(\boldsymbol{z}))\right] \ \text{(distortion)}.$$

Definition 7.1. The *rate–distortion function* is

$$R(D) = \inf_{q:\, \mathbb{E}d \le D} I(\boldsymbol{x}; \boldsymbol{z}).$$

$R(D)$ is convex and non-increasing, and gives the fundamental limit of lossy compression for the source $\boldsymbol{x}$ under distortion d.

7.2 Connection to autoencoders

The Lagrangian relaxation of the rate–distortion problem is

$$\min_{q} \; D + \beta R \;=\; \min_{q} \; \mathbb{E}\left[d(\boldsymbol{x}, \hat{\boldsymbol{x}}(\boldsymbol{z}))\right] + \beta\, I(\boldsymbol{x}; \boldsymbol{z}).$$

This is, term for term, the β-VAE objective. Sweeping β traces out the achievable (R, D) frontier (Alemi et al., 2018).

7.3 Information bottleneck (Tishby et al., 1999)

A different but related objective: given $(\boldsymbol{x}, \boldsymbol{y})$, find $\boldsymbol{z} = f(\boldsymbol{x})$ that is maximally informative about $\boldsymbol{y}$ and minimally informative about $\boldsymbol{x}$,

$$\min_{f} \; I(\boldsymbol{x}; \boldsymbol{z}) - \beta\, I(\boldsymbol{z}; \boldsymbol{y}).$$

Setting $\boldsymbol{y} = \boldsymbol{x}$ reduces to the unsupervised case, which is closely related to (and variational lower bounds of which give) the β-VAE (Alemi et al., 2017).

> **INTUITION** *Think of β as a knob that trades quality for bit-rate, like the JPEG quality slider. Small β means "spend bits freely, give me a faithful reconstruction". Large β means "be parsimonious, use as few bits as possible". In the middle is the RD frontier — the achievable best for each bit budget. Sweeping β does not just give you different models; it traces out the fundamental information-theoretic limit of your architecture on your data.*

Remark 7.2. Mutual information is notoriously hard to estimate in high dimension. The empirical claims about information-bottleneck dynamics during training (Tishby & Schwartz-Ziv 2017, Saxe et al. 2018) remain debated. Take them as inspiration, not measurement.

> **GOTCHA** *The KL term in a VAE is* not *an estimate of $I(\boldsymbol{x}; \boldsymbol{z})$. People often conflate them. The KL is an upper bound on the rate $I(\boldsymbol{x}; \boldsymbol{z}) \le \mathbb{E}_x[\mathrm{KL}(q(\boldsymbol{z}|\boldsymbol{x}) \,\|\, p(\boldsymbol{z}))]$, with equality only when $q(\boldsymbol{z}) = \mathbb{E}_x q(\boldsymbol{z}|\boldsymbol{x})$ matches the prior $p(\boldsymbol{z})$ exactly — which it usually doesn't. So your training-loss KL value overestimates how many bits you are actually using.*

> **EXAMPLE** **Sweeping beta traces the rate–distortion frontier**
>
> The cleanest demonstration of the rate–distortion view is to train the same VAE architecture on the same data with different values of β and measure both terms.
>
> **Setup.** dSprites dataset (Matthey et al. 2017): 737,280 binary 64×64 images of a single shape (square / ellipse / heart) varying in position, scale, and orientation. Six known ground-truth factors. Standard β-VAE architecture: 4 conv layers down, $d = 10$ latents, mirror decoder. Train each β for 3×10^5 steps.
>
> **Measured quantities.**
>
> | β | **Rate** (nats) | **Distortion** (BCE) | **Qualitative** |
> |---|---|---|---|
> | 0.5 | 28.4 | 0.092 | sharp recon, latent overuses dims |
> | 1.0 | 18.7 | 0.105 | standard VAE |
> | 4.0 | 11.2 | 0.138 | disentangled, partial mode coverage |
> | 16 | 4.6 | 0.187 | strongly disentangled, blurry recon |
> | 64 | 1.1 | 0.244 | near posterior collapse |
>
> Plot (R, D) pairs and you see the convex frontier of §7.2 in real measurement. Each β is a tangent slope; the curve they trace is the empirical $R(D)$ for this model class on this data.
>
> **What the rate buys you.** Rate is the average number of nats per image used to encode an example. At $\beta = 16$, the model uses ≈ 4.6 nats ≈ 6.6 bits — roughly enough to specify 100 distinct configurations. dSprites has $3 \times 6 \times 40 \times 32 \times 32 \approx 7 \times 10^5$ ground-truth configurations, so the model is heavily lossy and qualitatively learns the dominant factors of variation (position, scale) while collapsing the rest.
>
> **Practical takeaway.** You do not pick β once and for all; you sweep it for your task.

Reconstruction-first applications (compression, generation): $\beta \in [0.1, 1]$. Representation-first applications (downstream classification, disentanglement): $\beta \in [4, 32]$.

8 Variational autoencoders

8.1 Why probabilistic

The deterministic autoencoder we have so far has three concrete problems:

1. **No prior on $\boldsymbol{z}$.** The set of $\boldsymbol{z} = f(\boldsymbol{x})$ that the encoder produces is a strange, holey subset of $\mathbb{R}^d$. If you sample $\boldsymbol{z}$ uniformly or from a Gaussian, you decode to garbage.
2. **No likelihood.** We have a reconstruction error but no model of $p(\boldsymbol{x})$. Cannot compare models, cannot quantify uncertainty.
3. **Interpolations are not meaningful.** A straight line in $\boldsymbol{z}$ between two encodings need not decode to a smooth semantic transition.

The variational autoencoder (Kingma & Welling 2013; Rezende, Mohamed, Wierstra 2014) fixes all three by making the model probabilistic from the ground up.

8.2 The generative model

$$\begin{aligned} \boldsymbol{z} &\sim p(\boldsymbol{z}) = \mathcal{N}(0, I) \\ \boldsymbol{x} \mid \boldsymbol{z} &\sim p_\phi(\boldsymbol{x} \mid \boldsymbol{z}) \end{aligned}$$

The prior $p(\boldsymbol{z})$ is fixed — usually a standard normal — and the decoder defines a conditional likelihood $p_\phi(\boldsymbol{x} \mid \boldsymbol{z})$. The true posterior $p(\boldsymbol{z} \mid \boldsymbol{x})$ is generally intractable, so we introduce an *amortized* variational approximation $q_\theta(\boldsymbol{z} \mid \boldsymbol{x})$ — the "encoder".

8.3 Deriving the ELBO

We want to maximize $\log p_\phi(\boldsymbol{x})$. Multiply and divide by q_θ inside the integral:

$$\log p_\phi(\boldsymbol{x}) = \log \int p_\phi(\boldsymbol{x}, \boldsymbol{z})\, d\boldsymbol{z} = \log \int q_\theta(\boldsymbol{z} \mid \boldsymbol{x})\, \frac{p_\phi(\boldsymbol{x}, \boldsymbol{z})}{q_\theta(\boldsymbol{z} \mid \boldsymbol{x})}\, d\boldsymbol{z}.$$

The integral is an expectation over q_θ. Apply Jensen's inequality (log is concave):

$$\log p_\phi(\boldsymbol{x}) = \log \mathbb{E}_{q_\theta} \frac{p_\phi(\boldsymbol{x}, \boldsymbol{z})}{q_\theta(\boldsymbol{z} \mid \boldsymbol{x})} \;\geq\; \mathbb{E}_{q_\theta} \log \frac{p_\phi(\boldsymbol{x}, \boldsymbol{z})}{q_\theta(\boldsymbol{z} \mid \boldsymbol{x})} \;\equiv\; \mathcal{L}(\theta, \phi; \boldsymbol{x}).$$

Rearrange the right side using $p_\phi(\boldsymbol{x}, \boldsymbol{z}) = p_\phi(\boldsymbol{x} \mid \boldsymbol{z}) p(\boldsymbol{z})$:

$$\mathcal{L} = \mathbb{E}_{q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log p_\phi(\boldsymbol{x} \mid \boldsymbol{z})] - \mathrm{KL}\big(q_\theta(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\big).$$

INTUITION *The two terms pull in opposite directions. The reconstruction term wants $q(\boldsymbol{z}|\boldsymbol{x})$ to put a sharp spike on whichever $\boldsymbol{z}$ best decodes back to $\boldsymbol{x}$ — that minimizes distortion. The KL term wants $q(\boldsymbol{z}|\boldsymbol{x}) \approx p(\boldsymbol{z}) = \mathcal{N}(0, I)$ — a wide, vague distribution centred at the origin. Equilibrium is reached when the posterior is just informative enough to reconstruct $\boldsymbol{x}$ and no more. The KL is a* rubber band *pulling $q(\boldsymbol{z}|\boldsymbol{x})$ back toward the prior; reconstruction is the* force stretching it *toward an informative shape.*

8.4 Tightness of the bound

A short calculation:

$$\log p_\phi(\boldsymbol{x}) - \mathcal{L}(\theta, \phi; \boldsymbol{x}) = \mathrm{KL}\big(q_\theta(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p_\phi(\boldsymbol{z} \mid \boldsymbol{x})\big) \;\geq\; 0.$$

The bound is tight iff $q_\theta(\boldsymbol{z} \mid \boldsymbol{x}) = p_\phi(\boldsymbol{z} \mid \boldsymbol{x})$, i.e., the variational posterior equals the true posterior.

INTUITION *The amortization gap is the price of speed. A non-amortized variational method runs gradient descent on $(\boldsymbol{\mu}_i, \boldsymbol{\sigma}_i)$ for every datapoint $\boldsymbol{x}_i$ separately — slow at inference but tight. Amortized variational inference (the VAE encoder) replaces all of those with one shared network. At inference, you get a posterior in one forward pass; the cost is that the network cannot perfectly fit every posterior. Like training one student to take all the exams in a department — fast graduation, average grades.*

8.5 Closed-form KL for Gaussians

Take $q_\theta(\boldsymbol{z} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}_\theta(\boldsymbol{x}), \operatorname{diag} \boldsymbol{\sigma}^2_\theta(\boldsymbol{x}))$ and $p(\boldsymbol{z}) = \mathcal{N}(0, I)$. Then

$$\mathrm{KL}(q \,\|\, p) = \tfrac{1}{2} \sum_{i=1}^{d} \big(\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1\big).$$

8.6 The reparameterization trick

We want $\nabla_\theta \, \mathbb{E}_{q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log p_\phi(\boldsymbol{x} \mid \boldsymbol{z})]$. Rewrite the sample $\boldsymbol{z}$ as a deterministic, differentiable function of $\boldsymbol{x}$ and a parameter-free noise variable:

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I), \qquad \boldsymbol{z} = \boldsymbol{\mu}_\theta(\boldsymbol{x}) + \boldsymbol{\sigma}_\theta(\boldsymbol{x}) \odot \boldsymbol{\varepsilon}.$$

Now the expectation becomes

$$\mathbb{E}_{q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log p_\phi(\boldsymbol{x} \mid \boldsymbol{z})] = \mathbb{E}_{\boldsymbol{\varepsilon}}[\log p_\phi(\boldsymbol{x} \mid \boldsymbol{\mu}_\theta(\boldsymbol{x}) + \boldsymbol{\sigma}_\theta(\boldsymbol{x}) \odot \boldsymbol{\varepsilon})].$$

The randomness is now in $\boldsymbol{\varepsilon}$, which does not depend on θ. Gradients flow through $\boldsymbol{\mu}_\theta, \boldsymbol{\sigma}_\theta$ as ordinary backprop.

INTUITION *The trick separates "where" from "how much randomness". The model controls the location ($\boldsymbol{\mu}$) and scale ($\boldsymbol{\sigma}$) of the sample; nature contributes a fixed fair-coin source ($\boldsymbol{\varepsilon}$). When you ask "how does the loss change if I move $\boldsymbol{\mu}$ a tiny bit?", you can answer using the chain rule because the noise source is fixed — you have not changed which random draw you made, only what you scaled and shifted it by. Without this trick, moving $\boldsymbol{\mu}$ also re-samples $\boldsymbol{\varepsilon}$ implicitly, and you cannot tell which effect on the loss came from which change.*

8.7 Why it works: the Jacobian

The reparameterized sample $\boldsymbol{z} = T_\theta(\boldsymbol{x}, \boldsymbol{\varepsilon})$ has Jacobian with respect to θ given by the chain rule. The gradient

$$\nabla_\theta \, \mathbb{E}_{\boldsymbol{\varepsilon}}[\log p_\phi(\boldsymbol{x} \mid T_\theta(\boldsymbol{x}, \boldsymbol{\varepsilon}))] = \mathbb{E}_{\boldsymbol{\varepsilon}}[\nabla_\theta \log p_\phi(\boldsymbol{x} \mid T_\theta(\boldsymbol{x}, \boldsymbol{\varepsilon}))]$$

exchanges expectation and gradient (dominated convergence) and is an unbiased estimator. The variance is dramatically lower than the naive score-function estimator $\nabla \log q_\theta$ because the gradient path through T_θ carries far more information than the score.

EXAMPLE **The full MNIST VAE: every term in code, every term in numbers**

Take the MNIST AE from §4 and convert it. Same encoder up to the last layer; now the encoder produces $2d$ outputs — d means and d log-variances.

```
class VAE(nn.Module):
    def __init__(self, d=10):
        super().__init__()
        self.enc_body = ConvEncoder()            # to flat features
        self.mu       = nn.Linear(64*7*7, d)
        self.log_var  = nn.Linear(64*7*7, d)     # predict log sigma^2
        self.dec      = ConvDecoder(d)

    def forward(self, x):
        h = self.enc_body(x)
        mu, log_var = self.mu(h), self.log_var(h)
        std = (0.5 * log_var).exp()
        eps = torch.randn_like(std)
        z = mu + std * eps                       # reparameterization
        x_hat = self.dec(z)
        return x_hat, mu, log_var
```

```
def vae_loss(x, x_hat, mu, log_var):
    # Reconstruction: Bernoulli => BCE summed over pixels, mean over batch
    recon = F.binary_cross_entropy(x_hat, x, reduction='sum') / x.size(0)
    # Closed-form KL to N(0, I):  1/2 sum (mu^2 + sigma^2 - log sigma^2 - 1)
    kl = -0.5 * (1 + log_var - mu.pow(2) - log_var.exp()).sum() / x.size(0)
    return recon + kl, recon, kl
```

Numbers after 30 epochs ($d = 10$).

Term	Value (test, per image)
$-\mathbb{E}_q[\log p(\boldsymbol{x}\|\boldsymbol{z})]$ (reconstruction)	≈ 84.3 nats
$\mathrm{KL}(q(\boldsymbol{z}\|\boldsymbol{x}) \,\|\, p(\boldsymbol{z}))$	≈ 19.6 nats
$-$ELBO	≈ 103.9 nats
True $-\log p(\boldsymbol{x})$ (IWAE, $K = 5000$)	≈ 81.5 nats
Amortization gap	≈ 22 nats per image

What you can read off this.

- The bound is loose by $\sim$22 nats per image — a real gap, an active research problem.
- KL of 19.6 nats ≈ 28 bits per image is the model's effective code rate.
- Sample from $\boldsymbol{z} \sim \mathcal{N}(0, I)$ and decode: you get plausible-looking but blurry digits. The blur is the price you pay for the Gaussian decoder, not a bug in the training.

Posterior-collapse symptom (and fix). If you train with a strong autoregressive decoder (e.g. PixelCNN), the KL term often drops to < 1 nat: the decoder has learned to model $p(\boldsymbol{x})$ without using $\boldsymbol{z}$. The fix is KL warmup — multiply the KL term by a coefficient that ramps from 0 to 1 over the first few epochs:

```
beta_t = min(1.0, step / warmup_steps)      # 0 -> 1 over warmup_steps
loss = recon + beta_t * kl
```

This gives the decoder time to start using $\boldsymbol{z}$ before the regularization kicks in.

WHY THIS WORKS **The VAE in one paragraph.**

A VAE is a deterministic AE plus three modifications, each of which buys something concrete: (1) the encoder outputs a distribution $q(\boldsymbol{z}|\boldsymbol{x})$ instead of a point, which gives a smooth, densely-covered latent space that you can sample from; (2) the KL term pulls the aggregate posterior toward a prior, which makes that sample-from-the-prior strategy actually decode to realistic outputs; (3) the reparameterization trick makes the resulting objective trainable by ordinary backprop. The price you pay is the amortization gap (one network must approximate all posteriors) and the blurriness gap (Gaussian decoder averages over modes). Modern production VAEs — the ones inside Stable Diffusion — mostly suppress both by using small β (essentially deterministic AE with a tiny KL prior pressure) and a sharper-than-Gaussian reconstruction loss (LPIPS + adversarial). The probabilistic structure remains useful for the latent-space interface, but the model is no longer a serious likelihood model of pixels.

9 Modern variants

9.1 β-VAE

Reweight the KL term:

$$\mathcal{L}_\beta = \mathbb{E}_{q_\theta}[\log p_\phi(\boldsymbol{x} \mid \boldsymbol{z})] - \beta \,\mathrm{KL}(q_\theta \,\|\, p).$$

This is the rate–distortion Lagrangian. $\beta > 1$ encourages more factorized, axis-aligned codes at the cost of reconstruction quality; $\beta < 1$ does the reverse.

9.2 VQ-VAE: discrete latents

Maintain a codebook $\{\boldsymbol{e}_k\}_{k=1}^K \subset \mathbb{R}^d$ of learned vectors. Forward pass:

$$\begin{aligned}\boldsymbol{z}_e &= f_\theta(\boldsymbol{x}),\\ k^* &= \arg\min_k \|\boldsymbol{z}_e - \boldsymbol{e}_k\|^2,\\ \boldsymbol{z}_q &= \boldsymbol{e}_{k^*},\\ \hat{\boldsymbol{x}} &= g_\phi(\boldsymbol{z}_q).\end{aligned}$$

The arg min has no gradient, so use the *straight-through estimator*: in the backward pass, treat $\boldsymbol{z}_q$ as if it equals $\boldsymbol{z}_e$, i.e., copy gradients across the quantization step. Train with

$$\mathcal{L}_{\text{VQ}} = \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2 + \|\text{sg}[\boldsymbol{z}_e] - \boldsymbol{e}\|^2 + \beta\,\|\boldsymbol{z}_e - \text{sg}[\boldsymbol{e}]\|^2,$$

where sg$[\cdot]$ is the stop-gradient operator. The second term updates the codebook toward the encoder output; the third (commitment loss) updates the encoder toward its assigned code.

9.3 Other directions

- **Hierarchical VAEs** (NVAE, Very Deep VAE) chain latents $\boldsymbol{z} = (\boldsymbol{z}_1, \ldots, \boldsymbol{z}_L)$ with $p(\boldsymbol{z}) = \prod p(\boldsymbol{z}_\ell \mid \boldsymbol{z}_{>\ell})$. Approach exact likelihood SOTA on images.
- **Conditional VAEs** replace $p_\phi(\boldsymbol{x} \mid \boldsymbol{z})$ and $q_\theta(\boldsymbol{z} \mid \boldsymbol{x})$ by versions conditioned on side information $\boldsymbol{y}$. Useful for class-conditional generation and style control.
- **Tighter bounds.** The importance-weighted ELBO $\mathcal{L}_K = \mathbb{E} \log \frac{1}{K} \sum_{k=1}^K \frac{p(\boldsymbol{x}, \boldsymbol{z}_k)}{q(\boldsymbol{z}_k \mid \boldsymbol{x})}$ (Burda et al. 2016) is tighter for larger K, but the signal-to-noise ratio on encoder gradients degrades (Rainforth et al. 2018).
- **Flow posteriors.** Replace Gaussian q with a normalizing-flow posterior for richer shapes (Rezende & Mohamed 2015; Kingma et al. 2016).
- **Latent diffusion.** Train a VAE to compress images to a small latent, then run a diffusion model in latent space (Rombach et al. 2022). Stable Diffusion. Cuts compute 10–100× relative to pixel-space diffusion.

> **INTUITION** *VQ-VAE vs continuous VAE in one question: do you want your latent to be a vector or a sentence? Continuous VAEs give you a vector you can interpolate, add noise to, and follow gradients in — good for diffusion-style generation. VQ-VAEs give you a sequence of discrete tokens — good for autoregressive transformers that are built to predict the next token. The right choice depends entirely on what model consumes the latent next.*

> **INTUITION** *Why latent diffusion dominates: pixel-space diffusion solves the same problem twice. The diffusion model has to learn (a) low-level statistics like "pixels in a smooth region should be similar", and (b) high-level structure like "faces have two eyes". These are very different inductive biases. A VAE is well-suited to (a) — compression. A transformer or U-Net is well-suited to (b) — structure. Latent diffusion factors the problem cleanly between the two.*

> **EXAMPLE** **Three production systems built on AE backbones**
>
> **(1) Stable Diffusion's VAE.** Used in SD 1.x, 2.x, XL, and FLUX in various sizes. A KL-regularized VAE with patch-discriminator adversarial loss (Esser, Rombach, Ommer 2021).

Parameter	Value
Input resolution	$512 \times 512 \times 3$
Latent resolution	$64 \times 64 \times 4$
Spatial compression	8×8 per pixel
Channel compression	$3 \to 4$
Total compression ratio	48×
KL weight β	10^{-6} (very small)
Reconstruction loss	L1 + LPIPS + adversarial

The small β is deliberate: the VAE here is a near-deterministic compressor, not a generator. The diffusion model in the latent space is what does the generation.

(2) EnCodec (Défossez et al. 2022). Neural audio codec — the Spotify of VQ-VAEs. Used in MusicGen, AudioGen.

Parameter	Value
Input sample rate	24 or 48 kHz audio
Encoder downsampling	320× (24 kHz → 75 Hz tokens)
Residual VQ	8 codebooks × 1024 entries
Bitrate (configurable)	1.5, 3, 6, 12, 24 kbps
PESQ at 6 kbps	3.6 (Opus baseline at 12 kbps: 3.6)

EnCodec at 6 kbps matches Opus at 12 kbps — a 2× compression win from learning the codebook. The tokens are also the input vocabulary for generative audio models.

(3) MAE pretraining (He et al. 2022). The ViT-Huge config:

Parameter	Value
Input	224×224 ImageNet
Patch size	14×14 ($\rightarrow$ 256 patches per image)
Mask ratio	75%
Encoder	ViT-Huge, 632M params, sees visible patches only
Decoder	lightweight ViT, 220M params
Pretraining	1600 epochs on ImageNet-1k
Fine-tuning top-1	87.8%
Linear probe top-1	76.6%

The decoder is small and discarded after pretraining. The 75% mask ratio is deliberately aggressive: lower ratios let the encoder rely on local interpolation; higher ratios force genuine semantic representations.

What unifies these. All three use an AE as the compression interface between a modality (image, audio, image patches) and a representation suitable for further processing — diffusion in case 1, an autoregressive transformer in case 2, a downstream classifier in case 3. The AE is rarely the end product; it is almost always a component.

10 Applications

10.1 Anomaly detection

Train an AE only on normal data. Anomalies, being off-manifold, reconstruct poorly:

$$s(\boldsymbol{x}) = \|\boldsymbol{x} - g(f(\boldsymbol{x}))\|^2.$$

For VAEs, use $-$ELBO or the reconstruction probability. Beware Nalisnick et al. (2019): flow and VAE likelihoods are not reliable OOD detectors — they can be higher on OOD than in-distribution data. Combine with typicality tests or background-model likelihood ratios.

10.2 Restoration: denoising, inpainting, super-resolution

All three are special cases of supervised DAEs:

- **Denoising:** $\tilde{\boldsymbol{x}} = \boldsymbol{x} + \eta$, predict $\boldsymbol{x}$.
- **Inpainting:** $\tilde{\boldsymbol{x}} = M \odot \boldsymbol{x}$ with mask M, predict $\boldsymbol{x}$.
- **Super-resolution:** $\tilde{\boldsymbol{x}} =$ downsampled $\boldsymbol{x}$, predict $\boldsymbol{x}$.

Modern systems pair the AE with a learned prior (autoregressive or diffusion) over its latent for sharper, more diverse outputs.

10.3 Self-supervised pretraining: MAE and friends

He et al. (2022) train a Vision Transformer to reconstruct image patches given that 75% are masked at random. After pretraining, discard the decoder and use the encoder as a feature extractor. ViT-Huge with MAE pretraining reaches 87.8% ImageNet top-1 after fine-tuning — competitive with the best supervised pretraining.

The text-side analogue is BERT: mask tokens, reconstruct them. A DAE on text.

10.4 Generative modelling beyond images

- Audio codecs: EnCodec, SoundStream — VQ-VAEs at 1.5–24 kbps.
- Music: Jukebox — hierarchical VQ-VAE + transformer prior.
- Molecules: chemical VAE (Gómez-Bombarelli et al. 2018), junction-tree VAE for graphs — inverse design by gradient-based search in latent space.
- Scientific simulators: cosmological field emulators, particle-physics calorimeter shower simulation, astrophysical light-curve generation.

11 When to use an autoencoder (and when not to)

11.1 Reframing the question

The naive form is "autoencoder vs fully-connected layer (FCL)" or "autoencoder vs end-to-end supervised training". The first comparison is a category error — an autoencoder is a *training procedure*, an FCL is an *architectural primitive*; you can use FCLs as the encoder of an AE. The second is the real question:

Operational question. When does reconstruction-based pretraining (plus a small supervised head) beat end-to-end supervised training from scratch? And when does compression-via-AE beat operating in the ambient input space?

INTUITION *An AE is a way of trading two resources: you spend more unlabeled data and more unsupervised compute, and in return you save labeled data and downstream-task compute. If labeled data is cheap and downstream training is cheap, you cannot win this trade. If labels are expensive or downstream training has to be repeated, you can.*

11.2 Four conditions where AEs genuinely win

(1) Label scarcity ($N_{\text{unlab}} \gg N_{\text{lab}}$). The encoder learned by reconstruction has already paid the cost of modelling $p(\boldsymbol{x})$. Supervised fine-tuning only needs to learn the label-relevant projection on top. Empirically, MAE pretraining + linear probe reaches 76.6% on ImageNet with very limited fine-tuning data; training the same ViT from scratch on the same small labeled subset is dramatically worse. This is the strongest condition in practice.

(2) Multi-task downstream. One encoder, many downstream heads. Wav2Vec, CLIP, MAE, DINOv2 — pretrain once, deploy across dozens of tasks. End-to-end training would require a separate run per task; the AE amortizes the representation cost across them.

(3) Distribution shift / domain adaptation. Reconstruction loss is well-defined on unlabeled target-domain data; supervised loss is not. Pretrain on target, fine-tune the head on source labels — often beats joint training when source and target distributions differ.

(4) The downstream task is generation. If the deliverable is novel samples, you need a decoder. The AE is not a competitor to direct supervision here; it is the only thing that works. Latent diffusion is the cleanest example — you cannot do efficient text-to-image diffusion in pixel space at 512×512.

11.3 When NOT to use an autoencoder

GOTCHA *There are many ML problems where reaching for an autoencoder is just adding moving parts. Honest list of negative cases:*

- **Tabular regression / classification with adequate labels.** XGBoost or a supervised MLP wins, often decisively. The manifold hypothesis is weaker for tabular data, and gradient-boosted trees handle the mixed feature types better.
- **Time-series forecasting where labels are free.** The next value is the label. Direct supervised training (LSTM, Transformer, S4) usually beats AE-pretrain-then-forecast.
- **Image classification with full ImageNet labels.** Supervised ViT/ResNet match or beat AE-pretrained alternatives in many regimes. The AE advantage is in the label-scarce or transfer setting, not the labels-saturated one.
- **Settings where the encoder bottleneck destroys task-relevant information.** If your downstream task needs fine pixel detail (e.g. very-fine-grained classification), a compressive AE may discard the information your task depends on before the supervised head ever sees it.

11.4 What theory does and does not tell us

The linear regime is solved. Bourlard–Kamp (1988) and Eckart–Young (1936) give exactly the right answer: linear AE recovers the PCA subspace; reconstruction MSE equals the sum of discarded eigenvalues $\sum_{i>d} \sigma_i^2$. This is genuinely tight, but says nothing about downstream task performance.

Rate–distortion is a lower bound, not a guide. Any encoder/decoder pair operating at rate R has distortion $D \geq D^*(R)$, where D^* is the source's RD function. A trained AE traces a point above this curve. How far above depends on architecture and training — the theory does not tell you how to close the gap.

Information bottleneck is conceptually nice but not operational. The IB objective $\min I(\boldsymbol{x}; \boldsymbol{z}) - \beta\, I(\boldsymbol{z}; \boldsymbol{y})$ gives a principled definition of a "good" representation. But mutual information is notoriously hard to estimate in high dimension (McAllester & Stratos 2020), and the empirical claims about IB dynamics during training (Tishby & Schwartz-Ziv 2017) are disputed (Saxe et al. 2018). Take it as a framework, not a measurement.

Generic generalization bounds are vacuous. PAC-Bayes and NTK-based bounds applied to deep networks (including AEs) typically predict generalization gaps orders of magnitude larger than what is observed empirically. They do not distinguish AE pretraining from random initialization in any actionable way.

What you actually have is empirical and partial.

- *Erhan et al. (2010).* Pretraining acts as a data-dependent regularizer; the effect does not vanish at infinite labeled data (in their regime).
- *Saunshi et al. (2019, 2022).* For contrastive pretraining (close cousin), provable downstream gains under specific data-generating assumptions.
- *Lee et al. (2021).* Denoising-AE pretraining provably outperforms random initialization under structured-feature assumptions on the data.
- *Scaling laws.* Empirical, not theoretical, but the most useful predictor in practice: pretraining helps more as the unlabeled corpus grows relative to the labeled set.

WHY THIS WORKS **The honest state of theory.**

For the question "should I pretrain with an AE or train end-to-end?", theory gives you a framework (rate–distortion, information bottleneck) and a lower bound (you cannot beat the RD function). It does not give you a tight prediction of when AE pretraining helps a *specific* downstream task. The decision is empirical: run a small ablation. The framework tells you what to measure (rate, distortion, downstream metric); it does not tell you which model class will win.

---

12 Practical notes and pitfalls

12.1 Posterior collapse

Symptom. The KL term drives $q(\boldsymbol{z} \mid \boldsymbol{x}) \to p(\boldsymbol{z})$ and the decoder learns to ignore $\boldsymbol{z}$. Reconstruction becomes the unconditional mean.

Why it happens. Strong autoregressive decoders can model $p(\boldsymbol{x})$ without $\boldsymbol{z}$; the KL term then offers no benefit to using $\boldsymbol{z}$, so the encoder collapses.

Fixes. KL annealing (start with weight 0 and warm up), free bits (do not penalize the first λ nats of KL), β scheduling, weaker decoders, skip connections that force the decoder to use $\boldsymbol{z}$.

12.2 Blurry samples

A Gaussian decoder with pixel-MSE penalizes outliers, so it averages over plausible reconstructions. The cure is either a sharper likelihood (perceptual or adversarial loss) or a stronger prior over latents (VQ-VAE with autoregressive prior, latent diffusion).

12.3 Bottleneck sizing

Too small: underfitting, important variation lost. Too large with no regularization: identity map. Sweep d. Inspect the singular spectrum of the decoder Jacobian to estimate the effective latent dimension.

12.4 Evaluation

Reconstruction loss is not a generation metric. Use:

- FID for image samples,
- NLL bounds within a model family,
- downstream task accuracy for representation quality,
- held-out reconstruction error for anomaly detection.

12.5 Numerical hygiene for VAEs

- Predict $\log \sigma^2$, not σ. Clamp to a sane range.
- Use small initial KL weight to avoid $\log \sigma^2 \to -\infty$.
- Free bits stabilize early training.

13 Summary

1. An autoencoder is an encoder–decoder pair trained to reconstruct. The bottleneck and regularizers define what is modelled.
2. **Linear case = PCA.** The optimum is the truncated SVD subspace (Eckart–Young; Bourlard–Kamp). Identifiability is up to invertible reparameterization.
3. **Regularization** (sparsity, denoising, contraction) controls what off-manifold information is discarded. The geometric reading is via tangent and normal directions on $\mathcal{M}$.
4. **Denoising autoencoders learn the score** of the noise-smoothed data distribution (Vincent 2011). This bridge to score matching is the foundation of modern diffusion models.
5. **Information theory unifies many objectives.** Rate–distortion gives a principled reading of the β-VAE objective.
6. **VAEs** add a generative model and an amortized posterior, trained by maximizing the ELBO via the reparameterization trick.
7. **Modern systems** use AEs as compression backbones: VQ-VAE for tokenization, latent diffusion for generation, MAE for self-supervised pretraining.

Further reading

Foundations. Eckart & Young, *Psychometrika* (1936). Bourlard & Kamp, *Biol. Cybernetics* (1988). Baldi & Hornik, *Neural Networks* (1989). Olshausen & Field, *Nature* (1996). Hinton & Salakhutdinov, *Science* (2006).

Regularized AEs and score matching. Hyvärinen, *JMLR* (2005). Vincent et al., *ICML* (2008); *JMLR* (2010). Vincent, *Neural Computation* (2011). Rifai et al., *ICML* (2011). Alain & Bengio, *JMLR* (2014).

Variational AEs. Kingma & Welling, *ICLR* (2014). Rezende, Mohamed, Wierstra, *ICML* (2014). Burda, Grosse, Salakhutdinov, *ICLR* (2016). Higgins et al., *ICLR* (2017). Vahdat & Kautz, *NeurIPS* (2020).

Information theory. Shannon, *Bell System Tech. J.* (1948). Tishby, Pereira, Bialek, *Allerton* (1999). Alemi et al., *ICLR* (2017); *ICML* (2018).

Modern generative. van den Oord et al., *NeurIPS* (2017). Razavi et al., *NeurIPS* (2019). Song & Ermon, *NeurIPS* (2019). Ho, Jain, Abbeel, *NeurIPS* (2020). Rombach et al., *CVPR* (2022).

Pretraining and scientific ML. He et al., *CVPR* (2022). Locatello et al., *ICML* (2019). Gómez-Bombarelli et al., *ACS Central Science* (2018).